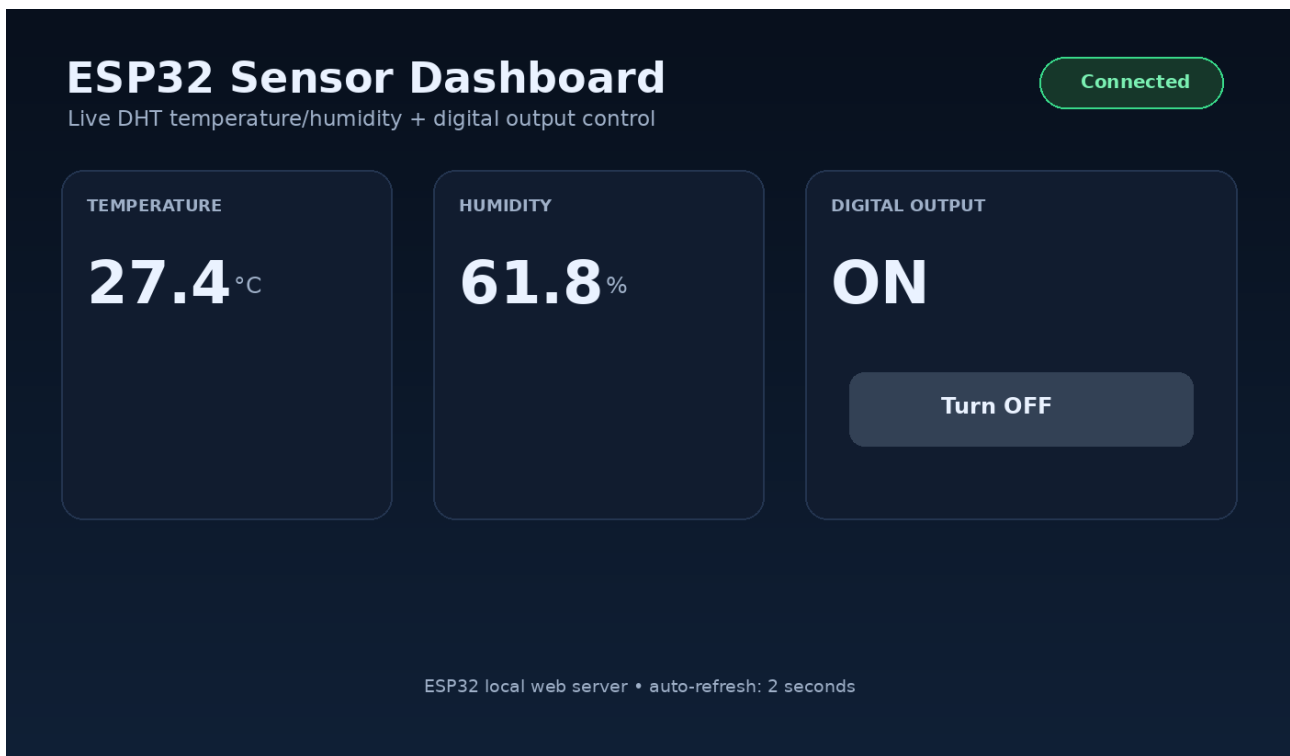


ESP32 LIVE SENSOR WEB DASHBOARD

Temperature & Humidity Monitoring with Digital Output Control



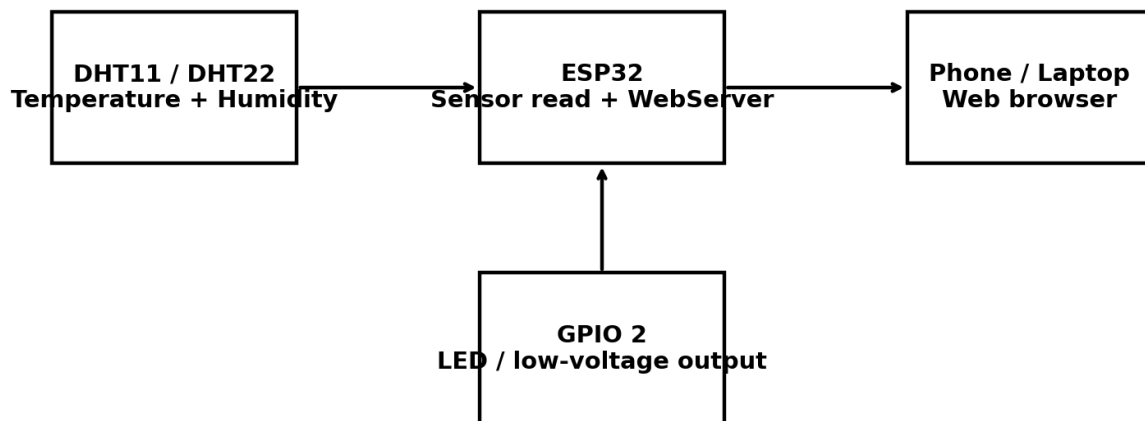
Deliverable: ESP32 sketch with embedded HTML/JavaScript, setup commands, wiring reference, dashboard preview, demo procedure, troubleshooting, and source links.

Target: ESP32 Arduino framework. Example sensor: DHT22; change one line for DHT11. Example output: low-voltage LED on GPIO 2.

1. Project Overview

This project turns an ESP32 into a small local web server. The board reads temperature and relative humidity from a DHT11/DHT22 sensor and serves a responsive dashboard to a phone or computer on the same Wi-Fi network. The browser polls a JSON endpoint for fresh sensor values and sends a POST request when the user toggles the digital output.

System architecture



Key behavior

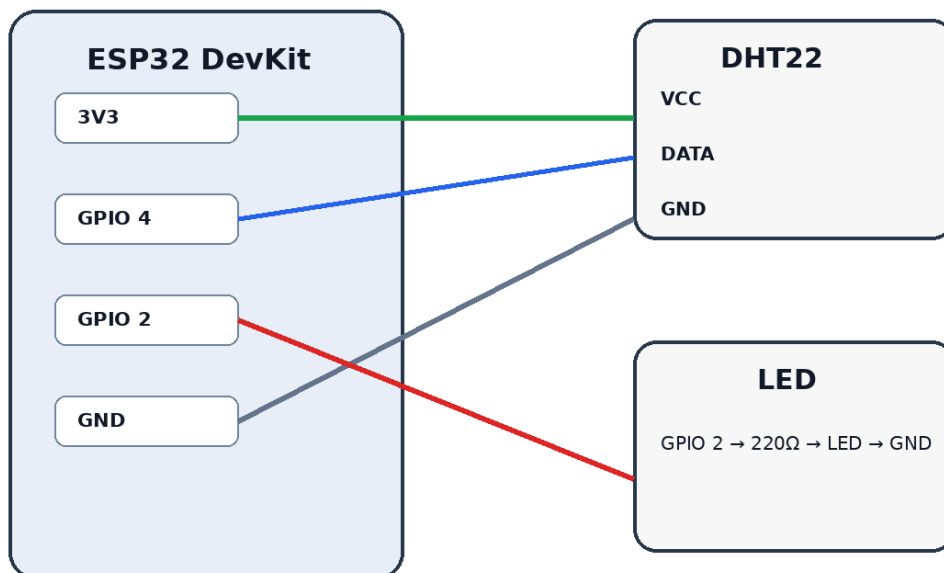
- ESP32 connects to Wi-Fi.
- DHT is sampled approximately every 2 seconds.
- The root URL serves embedded HTML/CSS/JavaScript.
- `/api/data` returns sensor data as JSON.
- `/api/toggle` changes the GPIO output.

Why this design? It is self-contained: no cloud account or separate server is required. The dashboard works locally while the ESP32 is powered.

2. Hardware and Wiring

Recommended setup: ESP32 development board, DHT22 or DHT11 module, one LED, one 220 Ω resistor, breadboard and jumper wires. The demonstration uses a low-voltage LED rather than mains switching.

Basic ESP32 + DHT22 + LED wiring



Use only low-voltage electronics for the demo; do not connect mains wiring.

Pin assignment

Function	ESP32 pin	Example connection
DHT VCC	3V3	Sensor VCC
DHT DATA	GPIO 4	Sensor DATA
DHT GND	GND	Sensor GND
LED output	GPIO 2	220 Ω resistor → LED → GND

3. Software Requirements and Installation

Install Arduino IDE (or Arduino CLI), add the ESP32 board package, and install the DHT libraries. The example uses the ESP32 Arduino WebServer class, so a separate async web-server library is not required.

Arduino IDE route

1. Install Arduino IDE.
2. Add the ESP32 boards package through Boards Manager.
3. Select your ESP32 board and port.
4. In Library Manager install **DHT sensor library** by Adafruit and **Adafruit Unified Sensor**.
5. Open the supplied ESP32_Sensor_Dashboard.ino.

Arduino CLI commands

```
arduino-cli core update-index
arduino-cli core install esp32:esp32
arduino-cli lib install "DHT sensor library"
arduino-cli lib install "Adafruit Unified Sensor"

arduino-cli compile --fqbn esp32:esp32:esp32 ESP32_Sensor_Dashboard
arduino-cli upload -p COM5 --fqbn esp32:esp32:esp32 ESP32_Sensor_Dashboard
# Linux example: /dev/ttyUSB0
```

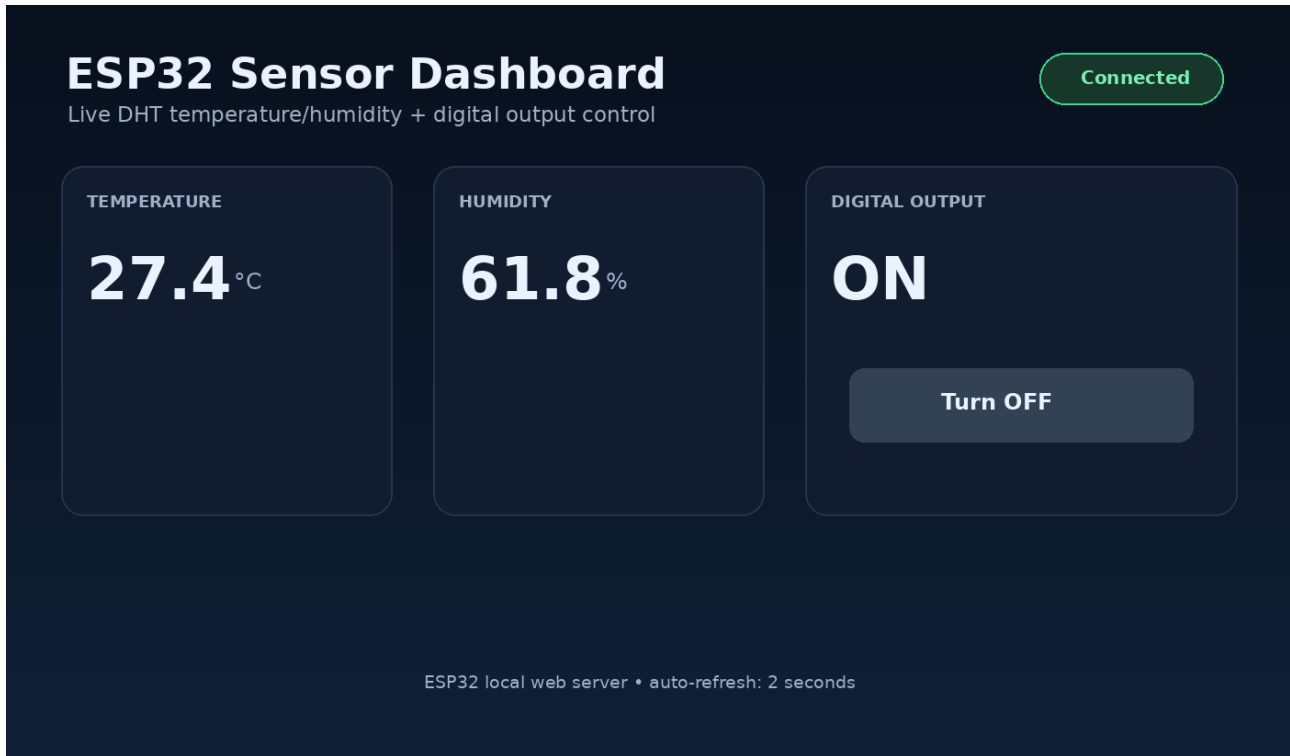
Replace YOUR_WIFI_SSID and YOUR_WIFI_PASSWORD in the sketch. Keep credentials private.

Official/reference links

ESP32 Arduino: <https://docs.espressif.com/projects/arduino-esp32/en/latest/>
Arduino IDE: <https://www.arduino.cc/en/software>
DHT library: <https://github.com/adafruit/DHT-sensor-library>
Unified Sensor: https://github.com/adafruit/Adafruit_Sensor

4. Dashboard Design

The page is embedded in firmware as a raw string. CSS creates a responsive card layout and JavaScript uses Fetch API to retrieve JSON without reloading the page.



Dashboard elements

- Temperature in °C.
- Humidity in %RH.
- Digital output ON/OFF and control button.
- Connection status badge.
- Automatic refresh every 2 seconds.

This is a generated dashboard preview matching the supplied HTML/CSS. For the final physical submission, capture a real browser screenshot after the ESP32 is online.

5. ESP32 Sketch — Core and Embedded Dashboard

```
/*
  ESP32 Live Sensor Dashboard
  Hardware: ESP32 + DHT11/DHT22 + LED
  Web server: ESP32 WebServer (built into ESP32 Arduino core)
*/

#include <WiFi.h>
#include <WebServer.h>
#include "DHT.h"

const char* WIFI_SSID = "YOUR_WIFI_SSID";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";

#define DHTPIN 4
#define DHTTYPE DHT22 // Change to DHT11 if required
#define OUTPUT_PIN 2 // External LED on GPIO 2

DHT dht(DHTPIN, DHTTYPE);
WebServer server(80);

float temperatureC = NAN;
float humidity = NAN;
bool outputState = false;
unsigned long lastReadMs = 0;
const unsigned long READ_INTERVAL_MS = 2000;

const char INDEX_HTML[] PROGMEM = R"rawliteral(
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width,initial-scale=1">
<title>ESP32 Sensor Dashboard</title>
<style>
:root { --bg:#0b1220; --card:#111c2f; --text:#eaf2ff; --muted:#9fb0c8; --accent:#39d98a; }
* { box-sizing:border-box; }
body { margin:0; font-family:system-ui,-apple-system,Segoe UI,Roboto,sans-serif;
background:linear-gradient(135deg,#08101d,#12243b); color:var(--text); }
.wrap { max-width:980px; margin:auto; padding:28px 18px 40px; }
header { display:flex; justify-content:space-between; gap:16px; align-items:center; flex-wrap:wrap; }
h1 { margin:0 0 5px; font-size:clamp(25px,5vw,40px); }
.sub { color:var(--muted); }
.grid { display:grid; grid-template-columns:repeat(auto-fit,minmax(220px,1fr)); gap:16px; margin-top:22px; }
.card { background:rgba(17,28,47,.92); border:1px solid #243550; border-radius:18px;
padding:20px; box-shadow:0 12px 30px rgba(0,0,0,.18); }
.label { color:var(--muted); font-size:14px; text-transform:uppercase; letter-spacing:.08em; }
.value { font-size:42px; font-weight:750; margin-top:8px; }
.unit { font-size:18px; color:var(--muted); }
button { width:100%; border:0; border-radius:12px; padding:13px 16px; margin-top:14px;
font-size:17px; font-weight:700; cursor:pointer; background:var(--accent); color:#062012; }
button.off { background:#334155; color:#eaf2ff; }
.badge { padding:7px 10px; border-radius:999px; background:#16372a; color:#7bf0b3; font-size:13px; }
footer { color:var(--muted); margin-top:20px; font-size:13px; text-align:center; }
</style>
</head>
<body>
<div class="wrap">
<header>
<div><h1>ESP32 Sensor Dashboard</h1>
<div class="sub">Live DHT temperature/humidity + digital output control</div></div>
<div class="badge" id="status">Connected</div>
</header>
<section class="grid">
<div class="card"><div class="label">Temperature</div>
<div class="value"><span id="temp">--</span><span class="unit"> °C</span></div></div>
<div class="card"><div class="label">Humidity</div>
<div class="value"><span id="hum">--</span><span class="unit"> %</span></div></div>
<div class="card"><div class="label">Digital Output</div>
<div class="value" id="out">OFF</div>
<button id="toggle" onclick="toggleOutput()">Turn ON</button></div>
</section>
<footer>ESP32 local web server • auto-refresh: 2 seconds</footer>
</div>
<script>
async function updateData() {
try {
const r = await fetch('/api/data', {cache:'no-store'});
const d = await r.json();
document.getElementById('temp').textContent =
Number.isFinite(d.temperature) ? d.temperature.toFixed(1) : '--';
document.getElementById('hum').textContent =
Number.isFinite(d.humidity) ? d.humidity.toFixed(1) : '--';
const on = d.output;
document.getElementById('out').textContent = on ? 'ON' : 'OFF';
const b = document.getElementById('toggle');
b.textContent = on ? 'Turn OFF' : 'Turn ON';
b.classList.toggle('off', on);
document.getElementById('status').textContent = 'Connected';
} catch(e) { document.getElementById('status').textContent = 'Connection error'; }
}
async function toggleOutput() {
await fetch('/api/toggle', {method:'POST'});
updateData();
}
updateData();
setInterval(updateData, 2000);
</script>
)rawliteral;
```

```

</script>
</body>
</html>
)rawliteral";

void readSensor() {
  if (millis() - lastReadMs < READ_INTERVAL_MS) return;
  lastReadMs = millis();
  float h = dht.readHumidity();
  float t = dht.readTemperature();
  if (!isnan(h) && !isnan(t)) {
    humidity = h;
    temperatureC = t;
    Serial.printf("Temperature: %.1f C | Humidity: %.1f %%\n", temperatureC, humidity);
  } else {
    Serial.println("DHT read failed; keeping last valid reading.");
  }
}

String jsonNumber(float value, int decimals) {
  if (isnan(value)) return "null";
  return String(value, decimals);
}

void handleRoot() { server.send_P(200, "text/html", INDEX_HTML); }

void handleData() {
  String json = "{";
  json += "\"temperature\": " + jsonNumber(temperatureC, 1) + ",";
  json += "\"humidity\": " + jsonNumber(humidity, 1) + ",";
  json += "\"output\": " + String(outputState ? "true" : "false");
  json += "}";
  server.send(200, "application/json", json);
}

void handleToggle() {
  outputState = !outputState;
  digitalWrite(OUTPUT_PIN, outputState ? HIGH : LOW);
  server.send(200, "application/json",
    String("{\"output\": " + (outputState ? "true" : "false")});
}

void setup() {
  Serial.begin(115200);
  pinMode(OUTPUT_PIN, OUTPUT);
  digitalWrite(OUTPUT_PIN, LOW);
  dht.begin();

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);

  Serial.print("Connecting to Wi-Fi");
  while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }

  Serial.println();
  Serial.print("Dashboard URL: http://");
  Serial.println(WiFi.localIP());

  server.on("/", HTTP_GET, handleRoot);
  server.on("/api/data", HTTP_GET, handleData);
  server.on("/api/toggle", HTTP_POST, handleToggle);
  server.begin();
  Serial.println("HTTP server started.");
}

void loop() {
  readSensor();
  server.handleClient();
}

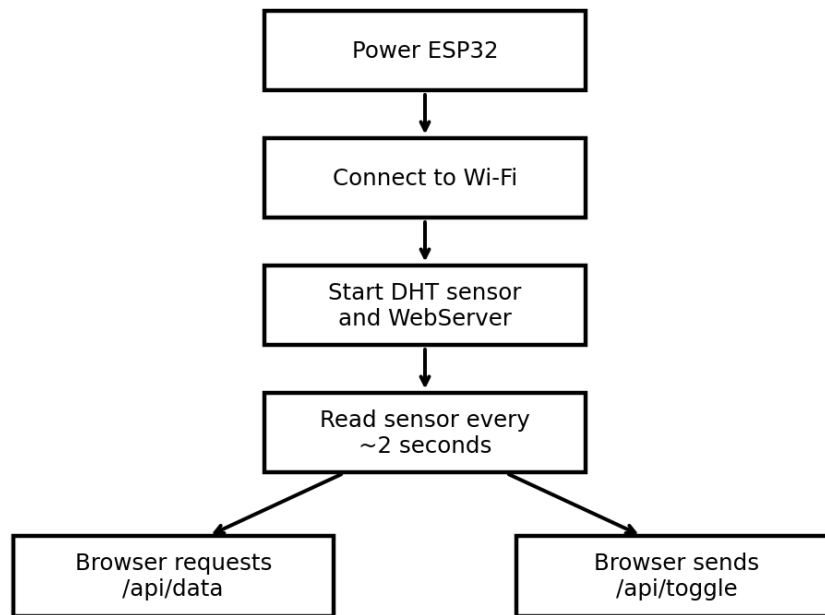
```

6. ESP32 Sketch — Server and Main Loop

The supplied .ino file is the complete runnable source. The loop uses `millis()` for the DHT interval so the HTTP server remains responsive.

7. Data Flow and HTTP Endpoints

Dashboard request / control flow



Method	Path	Purpose	Example
GET	/	Dashboard HTML	HTML page
GET	/api/data	Live state	{"temperature":27.4,"humidity":61.8,"output":true}
POST	/api/toggle	Toggle GPIO	{"output":false}

The browser does not need to know the GPIO number. It only uses the API, which keeps the UI separate from the hardware implementation.

8. Upload, Run, and Demo Procedure

1. Assemble the circuit.
2. Enter Wi-Fi credentials.
3. Select board and port.
4. Compile and upload.
5. Open Serial Monitor at 115200 baud.
6. Read the local dashboard IP.
7. Put the phone/laptop on the same Wi-Fi.
8. Open the IP in a browser.
9. Confirm live temperature/humidity.
10. Press the output button and verify the LED state changes.

Arduino Serial Monitor — example output

Connecting to Wi-Fi....

Dashboard URL: http://192.168.1.42

HTTP server started.

Temperature: 27.4 C | Humidity: 61.8 %

Temperature: 27.5 C | Humidity: 61.7 %

Temperature: 27.6 C | Humidity: 61.6 %

Temperature: 27.4 C | Humidity: 61.8 %

Demo notes

Explain that the ESP32 hosts the page itself. Show the IP address, then the browser dashboard, then the LED control. For the final submission, replace this example serial screenshot with a real capture from the board.

9. Testing and Expected Results

Test	Action	Expected result
Wi-Fi	Power with valid credentials	Serial Monitor prints local IP
Web page	Open IP	Dashboard loads
Temperature	Wait for update	Value updates
Humidity	Wait for update	Value updates
Output ON	Press Turn ON	LED turns on
Output OFF	Press Turn OFF	LED turns off
Sensor fault	Disconnect DATA briefly	Last valid value retained
Network mismatch	Use different Wi-Fi	Page is unreachable

Acceptance criteria

The project succeeds when the ESP32 boots without errors, prints a local IP, serves the dashboard, returns sensible DHT readings, and changes the GPIO output when the dashboard control is pressed.

10. Troubleshooting and Safety

DHT.h missing: install the Adafruit DHT Sensor Library and restart the IDE.

Dashboard unavailable: verify the IP and ensure the phone/laptop and ESP32 are on the same Wi-Fi. Some networks isolate wireless clients.

Values show --: check power, DATA wiring, DHT type, and GPIO. Allow a few seconds between DHT readings.

LED fails: check LED polarity, resistor, ground, and GPIO 2. If your board has a different built-in LED pin, use the external LED shown here.

Wi-Fi loops: verify SSID/password and access-point availability.

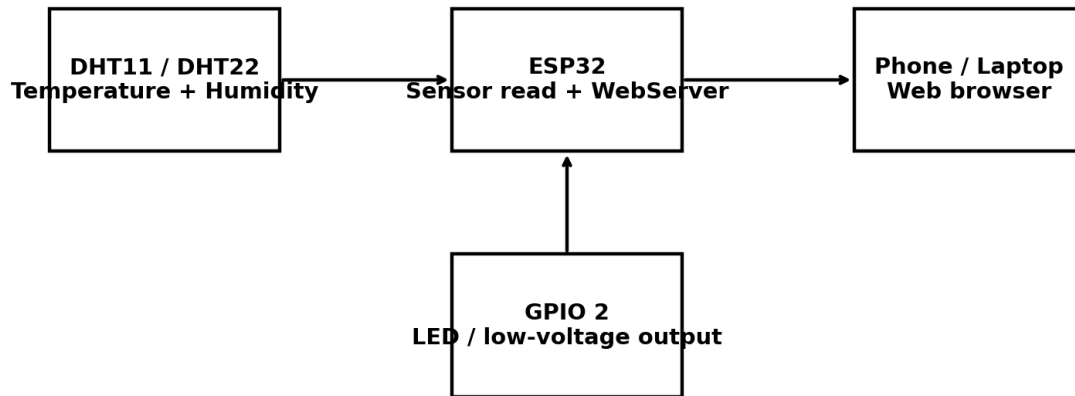
Safety note

Use low-voltage electronics only for this classroom prototype. If a relay module is later substituted, use an appropriate low-voltage module and follow its documentation. Do not connect household/mains voltage to this breadboard project.

11. Extensions and Future Work

Possible improvements include a browser-side history chart, refresh-now button, configurable temperature alert, uptime/RSSI fields, multiple outputs, another sensor, LittleFS-hosted web assets, and authentication for a trusted network.

System architecture



For an academic submission, demonstrate the required version first and present extensions as future work. Keep Wi-Fi credentials out of screenshots and public repositories.

12. Submission Checklist and References

Submission checklist

- ESP32_Sensor_Dashboard.ino source file
- PDF report
- Real dashboard screenshot from the ESP32
- Real Serial Monitor screenshot with assigned IP
- Hardware/wiring photo
- Demo video if required
- Final test observations

Reference links

1. Espressif Arduino-ESP32 documentation — <https://docs.espressif.com/projects/arduino-esp32/en/latest/>
2. Arduino software — <https://www.arduino.cc/en/software>
3. Adafruit DHT Sensor Library — <https://github.com/adafruit/DHT-sensor-library>
4. Adafruit Unified Sensor Library — https://github.com/adafruit/Adafruit_Sensor

The report's library guidance follows the Adafruit DHT library and Unified Sensor documentation, and the ESP32 portion follows the Arduino-ESP32 framework.

End of report